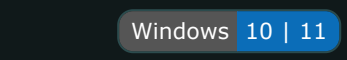
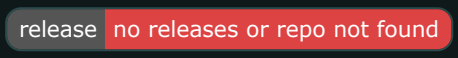
Audion Hub Manager





Version 1.10.0 · 2026-08-24 · 181.3 MB

- Direct download — unmetered, no rate limits
- Project page — every version and how to install

`SHA-256: 4c6e8f5bdf19405b0d2bf9b99243a9a32cecce986654ca40d4134227d00bda3b`

Audion Hub Manager is a local portable/NiceGUI workbench for project

projection, Markdown documentation, Git, VS Code and safe mirroring.

Core model:

```
Source = full project and source of truth

Hub Data = filtered technical mirror with Git

Docs = optional human-readable documentation layer
```

The tool makes large projects visible, reviewable, committable and recoverable

without copying runtime noise, caches, logs, build artifacts or local secrets.

Project Selection Model

The `Project` dropdown selects an entry from `config/projects.json`. It is not

a tree filter and it is not a folder picker inside the currently visible tree.

It switches the whole active project bundle:

```
source_path     -> Project/Source layer
projection_path -> Hub Data / Mirror layer
docs_path       -> Docs layer
profile         -> MIRROR rules and commit allowlist
default_branch  -> branch used by Git commands
```

Relative paths in `projects.json` are resolved from the Hub Manager root. This lets a portable bundle use `"source_path": "."` for itself and short relative paths for projects inside the same moved tree. When a selected path is outside the manager tree, Hub Manager keeps the absolute path.

The `Structure` panel has three layer switches for the selected project. The badge near the title shows the active layer and its path:

```
Project = live source_path of the selected project
Mirror  = filtered Hub Data mirror for that project
Docs    = readable Markdown/text folder for that project
```

If one parent folder contains many projects, such as `S:/TOOLS/Apps/`, do not turn that parent into one project. Use `Rebuild dropdown` / `Scan projects`: Hub Manager detects real nested project roots, including double folders such as `Project/Project`, and

adds separate entries to `projects.json`. After that, switch projects one by one in the dropdown and run MIRROR/commit for the active entry.

If the registry contains stale demo/sample records, missing `source_path` values or duplicates, use `Support -> Clean projects.json`. It cleans only `config/projects.json`; it does not delete Source, Hub Data, Docs or project folders.

Documentation

- `docs/README_RU.md` / `docs/README_EN.md` — project overview.
- `docs/USER_GUIDE_RU.md` / `docs/USER_GUIDE_EN.md` — user workflow.
- `docs/TECH_SPEC_RU.md` / `docs/TECH_SPEC_EN.md` — technical specification.
- `docs/GIT_WIKI_HUB_MANAGER_RU.md` / `docs/GIT_WIKI_HUB_MANAGER_EN.md` — detailed Git-work wiki.
- `AGENTS_RU.md` / `AGENTS_EN.md` — Codex/agent contract.
- `Docs/` — user and strategic documentation.
- PDF copies are generated only on explicit request and are not stored as a normal docs mirror.

Markdown is the primary format. PDF output is an explicit one-off export, not a tracked duplicate documentation tree.

Features

- Load project registry from `config/projects.json`.
- Scan a parent folder with many projects and add detected projects to the registry.
- Show a `SOURCE` badge for all registered projects after `Batch Git status`: `projects / clean / dirty / errors`.
- Run `SOURCE ACTIONS` for the registry: rebuild dropdown, batch preview MIRROR, Clone Source, batch safety, batch verify, batch Git status and

combined workspace.

- Run `PROJECT ACTIONS` for the selected tree object: refresh, load to Editor, open in VS Code, copy relative/full path.
- Open the current project through `Open Project`, plus Mirror, Docs Folder, VS Code, Terminal and Git. `Terminal` opens a Source terminal; `Git` opens a Git-root terminal and immediately runs `git status --short`.
- Clean missing Source paths and duplicate records from `projects.json` through `Support -> Clean projects.json`.
- Build Hub Projection using `config/projection_profiles.json`.
- Run dry-run and apply MIRROR with Source protection.
- Show Project/GIT COPY/Docs tree layers in the `Structure` panel.
- Run transparent Git commands and display exact terminal output.
- Cover almost the full everyday Git lifecycle: init/status/inspect, diff, stage/restore, commit, tag, remote sync, branch/switch, stash, history/graph, recovery, maintenance, clone into Source and the manual command cache. Rare or dangerous operations remain explicit templates.
- Keep the main local Git commands in `Quick -> GIT LOCAL`: `init`, `status`, `root`, `log --oneline` and `reflog`. `user config` lives in the Auth block inside `Remote`, `config list` lives in Git backup/maintenance, and graph history lives in the `History` pane.
- Use `Branch` for branch status, `branch -vv`, `switch`, `switch -c`, tags, compare branches, `merge --no-ff`, `revert`, `cherry-pick`, stash and abort/rebase templates.
- Prepare readable commit/checkpoints through Basket.
- Open Markdown/text files in the embedded Editor with icon-only toolbar:

load/save/paste/copy/clear/VS Code/expand.

- Show the two-row right tab grid: Quick, Branch, Editor, Diff, Storage;

  Remote, Basket, Reader, History and Details. Safety Scan lives in Storage.
- Use Material icons in the right tab headers and delayed tooltips on command

  buttons and tabs.
- Use `Remote` for remote URL construction, recent field caches,

  `push origin`, `pull --ff-only`, `fetch --all --prune`, `remote -v`,

  `push all remotes`, `apply remotes.json`, `origin push URLs` and Auth setup.
- Check the BLAKE3 backend and run read-only Verify Mirror for Source <-> Hub Data.
- Probe external Git authentication without storing tokens.
- Help recover a project from Hub Data if Source was deleted.

Start

GUI:

```
launcher_gui.cmd
```

CLI smoke:

```
launcher_cli.cmd --mirror-preview demo_local --json

launcher_cli.cmd --mirror-apply demo_local --json

launcher_cli.cmd --mirror-apply demo_local --apply --json
```

If the portable runtime is missing:

```
install\Build_Portable_Env.cmd
```

Project Setup

Projects are defined in:

```
config/projects.json
```

Example:

```
{
  "id": "audion_hub_manager",
  "title": "Audion Hub Manager",
  "source_path": ".",
  "projection_path": "S:/Audion/Hub Data/Audion Hub Manager",
  "docs_path": "S:/Audion/Docs/Projects/Audion Hub Manager",
  "profile": "audion_python_project_projection",
  "default_branch": "main"
}
```

`source_path` is the live full project. Relative values are resolved from the

Hub Manager root. MIRROR must not write there.

`projection_path` is the filtered Hub Data mirror. It can be rebuilt and can

have its own `.git`.

`docs_path` is an optional docs-view for Markdown/text reading.

Safety Rules

1. Source is the source of truth.
2. Hub Data is derived and rebuildable.
3. MIRROR does not modify Source.
4. `.git/**` is protected.
5. Real MIRROR apply requires explicit intent.
6. Hub Manager commits must use the same allowlist as the active Hub profile.
7. Do not store tokens, passwords, private keys or local paths in shared config.
8. Keep machine-local settings in excluded files such as `*.local.json` or `.env`.
9. BLAKE3 manifests/check reports are written only to `logs/<project_id>/`.
10. Docs/Obsidian/LogSeq/VS Code docs folders are not hashed separately: their canonical technical copy already lives in Hub Data.

`Verify Mirror` is a read-only Source ↔ Hub Data check for the active profile.

It builds manifests in memory and writes one timestamped JSON result to `logs/<project_id>/`. It does not create manifest files inside Source, Hub Data or Docs.

Git

Recommended model:

```
Full Project Source/
  .git/

Hub Data/<project>/
  .git/
```

Source Git supports live development, VS Code and agents.

Hub Git is a clean history for the filtered technical mirror.

Hub Manager should not run broad `git add .`. It should stage/commit only files accepted by the active projection profile.

Remote And Auth

Hub Manager is not a secrets vault.

Remote names, repository owners/groups and repository names can be entered in the `Remote` pane. The recent-value selects cache up to 20 values per field and apply a value only after explicit selection, so typed or pasted input is not overlaid by cache text.

Use external tools:

```
gh auth login

glab auth login

ssh -T git@github.com

ssh -T git@gitlab.com
```

HTTPS credentials belong in Git Credential Manager or the OS credential store.

SSH keys belong to SSH/ssh-agent.

Platform and URL type are chosen with buttons in the top row of `Remote`, not with dropdowns. `Forgejo`, `Gitea` and `Custom host` additionally show server address and SSH port fields; known instances live in `config/forgejo_hosts.json`, which holds no tokens.

A Forgejo/Gitea account is connected the ordinary way for those servers: a personal access token from `Settings -> Applications`. Hub Manager verifies the token against `/api/v1/user` and hands it to your Git credential helper; it is never written into the project files. From then on `git push` reads the token from the store by itself, while the API buttons show the account, list repositories and can create a new one. Rationale and details are in `docs/GIT_AUTH_STRATEGY.md`.

Checks

```
python -m compileall -q system_core

python -m pytest -q tests

python system_core\ui_nicegui\app.py --smoke
```

Documented baseline: `121 passed`.

Recovery

If Source is deleted but Hub Data survived:

```
robocopy "<Hub Data>\Audion Hub Manager" "<portable-root>\Audion Hub Manager" /E
/COPY:DAT /DCOPY:DAT /R:1 /W:1 /XJ
```

Then rebuild runtime or release artifacts if needed. Generate PDF only for an explicit handoff/archive request.

Project formula:

```
Source stays whole.

Hub stays reviewable.

Docs stays readable.

Git stops being scary.
```